

ITI | April 2026

React

Islam Baidaq | Software Architect @ Vodafone

Routing & Navigation

Nested Routes

Layout reuse

```
<Route path="/dashboard" element={<Dashboard />}>  
  <Route path="profile" element={<Profile />} />  
  <Route path="settings" element={<Settings />} />  
</Route>
```

Navigation & Redirects

UseNavigate

```
import { useNavigate , Navigate } from "react-router-dom";  
  
const navigate = useNavigate();  
navigate("/home");
```

```
<Navigate to="/login" />
```

Routing & Navigation

Protected Routes

Basic Protected Route

```
const ProtectedRoute = ({ children }) => {  
  const isAuthenticated = false;  
  return isAuthenticated ? children : <Navigate to="/login" />;  
};
```

Usage

```
<Route  
  path="/dashboard"  
  element= {  
    <ProtectedRoute>  
      <Dashboard />  
    </ProtectedRoute>  
  }  
</Route>
```

Forms & Validation

Controlled Inputs

1. What is a Controlled Input?

React controls the input value

```
const [name, setName] = useState("");  
  
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)}  
>
```

2. Why Controlled?

- React is source of truth
- easier validation
- predictable behavior

Forms & Validation

Controlled Inputs

3. Multiple Inputs

```
const [form, setForm] = useState({  
  email: "",  
  password: ""  
});
```

4. Generic Handler

```
const handleChange = (e) => {  
  setForm({  
    ...form,  
    [e.target.name]: e.target.value  
  });  
};
```

Forms & Validation

Validation Basics

```
const [error , setError] = useState("");

if (!email.includes("@")) {
  setError("Invalid email");
}

{error && <p>{error}</p>}
```

Problems:

- too much state
- too many handlers
- hard to scale

Forms & Validation

Forms

- React Hook Form (self)
- React Formik

1. Install Formik

```
npm install formik yup
```

2. Basic Formik Usage

```
import { useFormik } from "formik";

const { values } = useFormik({
  initialValues: {
    email: "",
    password: ""
  },
  onSubmit: (values) => {
    console.log(values);
  }
});
```

```
<input
  name="email"
  value={values.email}
  onChange={handleChange}
/>
```

Forms & Validation

Forms

```
import * as Yup from "yup";

const schema = Yup.object({
  email: Yup.string().email("Invalid email").required("Required"),
  password: Yup.string().min(6, "Min 6 chars").required()
});
```

```
const { values, errors } = useFormik({
  initialValues: { email: "", password: "" },
  validationSchema: schema,
  onSubmit: ...
});
```

```
{ errors.email &&
  <p>{errors.email}</p>
}
```

Forms & Validation

Mini Project : Auth Forms (Formik + Yup)

Features:

- login form
- signup form
- validation with Yup
- error messages
- redirect after submit

Global State Management

1. Types of State

- Local state (input value)
- Shared state (theme)
- Server state (API data)

Problem is : Passing props through all levels 🥵

When Global State is Needed

- authentication
- theme (dark/light)
- user data

Global State Management

Context API

Share data without prop drilling

1. Create Context

```
import { createContext } from "react";

export const ThemeContext = createContext();
```

2. Provider

```
import { useState } from "react";
import { ThemeContext } from "../ThemeContext";

const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState("light");

  return (
    <ThemeContext.Provider value={{ theme }}>
      {children}
    </ThemeContext.Provider>
  );
};

export default ThemeProvider;
```

3. Use context

```
import ThemeProvider from "../ThemeProvider";

const App = () => {
  return (
    <ThemeProvider>
      <MainApp />
    </ThemeProvider>
  );
}
```

```
import { useContext } from "react";
import { ThemeContext } from "../ThemeContext";

const { theme } = useContext(ThemeContext);
```

Global State Management

Context Limitations

Problems:

- re-renders all consumers
- not scalable for large apps
- messy structure

So If we scale:

```
<ThemeProvider>  
  <AuthProvider>  
    <CartProvider>  
      <NotificationProvider>  
        <App />  
        .....  
.....
```

😓 messy tree

Global State Management

Redux

- Single store
- Centralized logic
- Predictable updates

Redux Toolkit Setup

```
npm install @reduxjs/toolkit react-redux
```

Create Store

```
import { configureStore } from "@reduxjs/toolkit";  
  
export const store = configureStore({  
  reducer: {}  
});
```

```
import { Provider } from "react-redux";  
import { store } from "../store";
```

```
<Provider store={store}>  
  <App />  
</Provider>
```

Global State Management

Slices (Core Concept)

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment: (state) => {
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    }
  }
});

export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

Update Store

```
import counterReducer from "../counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

Global State Management

Using Redux in Components

useSelector

```
import { useSelector } from "react-redux";  
  
const count = useSelector((state) => state.counter.value);
```

useDispatch

```
import { useDispatch } from "react-redux";  
import { increment } from "../counterSlice";  
  
const dispatch = useDispatch();  
  
<button onClick={() => dispatch(increment())}>  
+  
</button>
```

Global State Management

(Auth + Protected ui + Dashboard + Cart system)

Exercises

- Login/logout flow
- Show username in navbar
- Show cart count in navbar
- Dashboard which contain list of products
- Crud over them

Global State Management

Server state ::: Async Logic (RTK Thunk)

createAsyncThunk

```
import { createAsyncThunk } from "@reduxjs/toolkit";

export const fetchUsers = createAsyncThunk(
  "users/fetchUsers",
  async () => {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    return res.json();
  }
);
```

Handle in Slice

```
extraReducers: (builder) => {

  builder
    .addCase(fetchUsers.pending, (state) => {
      state.loading = true;
    })
    .addCase(fetchUsers.fulfilled, (state, action) => {
      state.loading = false;
      state.users = action.payload;
    })
    .addCase(fetchUsers.rejected, (state) => {
      state.loading = false;
    });
}
```